```
const EXPECTED_BODY = { id: 1, name: "My movie", year: 1999 };
```

Feel free to update `EXPECTED_BODY` to be of any value, as long as the data type matches. Using Pact matchers, we're saying that the `EXPECTED_BODY` should have an ID, the movie name should be something similar to "My movie", and the year should be something similar to the year 1999.

Remember that with contract testing, the consumer should care more about the shape of the data rather than the data itself. This is what we refer to as loose matching. We use the `MatchersV3.eachLike` matching rule to ensure the type matches all the values. This way, the test should still pass if the provider returns different values, but the types are similar. If, for whatever reason, the movie ID changes from an integer to a string, the matching rule will fail, and the contract test will throw an error.

CALLING THE CONSUMER AGAINST THE MOCK PROVIDER

The next step is to call the function the consumer created as responsible for fetching all the movies. In this example, the `fetchMovies` function is passed in as a callback function to `provider.executeTest`. A callback function is a function that is passed as an argument to another function executed by that function:

```
await provider.executeTest(async mockProvider => {
    const movies = await fetchMovies(mockProvider.url);
})
```

The `provider.executeTest` function provides a dynamic mock provider that is passed into `fetchMovies`. One available property of the mock provider is the URL itself, which we can pass into `fetchMovies`. Instead of the actual movies API URL, the `fetchMovies` will be tested against the URL of the mock provider, which could require some refactoring to make it more testable.

VERIFY THE TEST AND CONTRACT GENERATION

Once the `fetchMovies` function is called, we need to verify that the actual data returned by the consumer matches the expected data. We do this by comparing the actual data returned by the consumer and the expected data